

Class Notes: Neural Network Architecture

1. What are the building blocks for a neural architecture?

Convolution: `nn.Conv2d`

Extracts features

Maxpooling: `nn.Maxpool2D`

Reduces feature dimensions

Linear : `nn.Linear`

Classifier

a) Can I build a classifier without conv and maxpool?

Yes, I can only use linear, but convolution may make my features more unique and give me better performance, but remember to convert your image to a 1D vector using `nn.Flatten`

b) Is it absolutely necessary for me to use maxpool with conv?

You can use averagepool instead of maxpool. You can completely ignore maxpool if you have small image inputs (say 20 X 20). Its better to use maxpool if image sizes are larger (say 200 X 200)

2. Looking at convolution operation in detail

Parameters:- kernel/window/filter/mask , stride, dilation, padding, no. of filters

Image: 200 * 200. kernel 3 * 3, no padding: out: 198*198

Image: 200 * 200. kernel 5 * 5, no padding: out: 196*196

Image: 200 * 200. kernel 3 * 3, 1 padding (on all 4 sides): out: 200*200

Image: 200 * 200. kernel 5 * 5, 2 padding: out: 200*200

a) Is it necessary to use a odd size kernel, 3*3, 5*5

No, but centering is easy with odd

b) Is it necessary to use a square kernel?

No, but we want to have same number of neighbours in the height and width.

c) Is padding necessary?

Not if my goal is just feature extraction.

d) How to decide kernel size? 3×3 or 11×11 ?

It depends on whether I want a small neighborhood or larger.

e) When to reduce features with stride, when with dilation?

Both reduces number of features. Stride ignores some regions assuming they are the same as the previous. Dilation enlarges the neighborhood without changing the number of neighbors. When I need a larger spatial neighborhood without increasing the number of neighbors (parameters), I will go for dilation.

3. Looking into actual networks

a) Alexnet

b) VGG

c) Resnet

d) Inception

more conv $\rightarrow$ more features $\rightarrow$ better networks

however

more features $\rightarrow$ more parameters $\rightarrow$ complex networks

more parameters but less data $\rightarrow$ unstable solution

4. The main kind of blocks

a. Convolution

b. Depth Convolution

c. Attention

d. Residual

e. Multiscale

- Can I use linear blocks in between convolution blocks:- Not directly
They are usually at the end for classification

-Can I use maxpool after every layer- Yes, they will keep reducing the scale

b) Lets start with depth convolution

input:-224*224

A normal convolution single filter of 3*3*3 volume on a 224 *224 image without padding gives me 222*222*1

normal convolution

```
self.conv1=nn.Conv2d( in=3, out=32, kernel_size=3, padding=0, stride=1, dilation=1)
```

```
x=self.conv1(x)
```

222*222*32 , $3*3*3 * 32 = 864$ (params)

or depth convolution

```
self.conv1=nn.Conv2d( in=3, out= 32, kernel_size=3, padding=0, stride=1, dilation=1,groups=?)
```

A depth convolution single filter of 3*3*3 volume on a 224 *224 image without padding gives me 222*222*3

```
x=self.conv1(x)
```

222*222*3 $(3*3*3 = 27)$

```
self.conv2=nn.Conv2d( in=3, out= 32, kernel_size=1, padding=0, stride=1, dilation=1)
```

222*222*32 $(1*1*3*32 =96)$

$$=27+96 = 123 \text{ (params)}$$

c) Attention Block:-

channel attention

my input $x = 222 \times 222 \times 32$ (32 channels)

i) reduce the $222 \times 222 \times 32$ into $1 \times 1 \times 32$

ii) computing linear weights $\text{nn.Linear}(32, 16)$, $\text{nn.Linear}(16, 32)$
 1×32 (32×16) 1×16 (16×32)

$\text{prob} = \text{sigmoid}(1 \times 32)$, it will give me 32 prob between 0-1

my final output $x * \text{prob}$

32×16 , 16×32 are the weights learnt during training

$\text{new_weights} = \text{old_weights} - \text{learning_rate} * \text{derivative}(\text{error})$, where error is $\text{actual_output} - \text{predicted_output}$

d) Take a look at fig 3 (last figure) of <https://arxiv.org/pdf/1512.03385>

e) multiscale:

$224 \times 224 \times 3$

- $224 \times 224 \times 32$ (1×1)
- $224 \times 224 \times 32$ (3×3 , padding=1)
- $224 \times 224 \times 32$ (5×5 , padding=2)

output:-224*224*96

What do we know so far?

- a)How to define a problem and approach its solution for a CV task.
- b)How to use Convolution in Neural Network, its role and variabilities.
- c)When to solve using transfer learning and when to do finetuning
- d)Different blocks in neural network and factors (feature size, parameters) as well as behavior of gradients in particular cases.

Approach :-

- I) Any task will be described , you need to formulate it (ex. Lab 2)
- II) Given a sample model, being able to modify and comment on it in terms of feature size, parameters etc.

Augmentation:-

blur:- cv2.blur , this is just a sliding windowed mean operation

noise:- adding a normal distribution matrix same size as the image with the image

rotation:-

brightness:- add pixels to the image

contrast:- multiply

12, 30 -18

24 60 - 36

check log and gamma operations

Activation:-

In any kind of networks to add nonlinearity to the features:-ReLU, PReLU, ELU, SeLU

Can be added after each conv set

Helps solve the vanishing gradient problem

While ReLU has a strict rule, the others soften it dealing with dying neuron problem.

Tanh, sigmoid, HardSigmoid are typically used at the end of the networks

Sigmoid is basically weighting- so if you need to weight inbetween the network(say channel attention, you may use it)

But remember, ReLU group works directly on the feature.

Sigmoid group in between layers is used to weight features so typically

`out= nn.ReLU(x)`, x is input features

never use

`out= nn.Sigmoid(x)` inbetween layers, rather

`out = x * nn.sigmoid(f1(x))`

Optimizers

Lets take a sample 2 layered network

```
class 2lnet:
    def __init__():
        self.l1=nn.Linear(20,5)
        self.l2=nn.Linear(5,3)

    def forward():
```

```

x1=self.l1(x)
x2=self.l2(x1)
return x2

```

Mathematically

```

x1= x.W1
x2=x1*W2

```

If target is o

Loss (assume mean square error) $L = (o-x2)^2$

For my update of W1 and W2, I will require $dL/dW1$ and $dL/dW2$

```

W1t = W1t-1 - learning_rate* dL/dW1t-1
W2t = W2t-1 - learning_rate* dL/dW2t-1

```

My function is $L = (o-x2)^2$

- a. find $dL/dx2 = -2(o-x2).1$*
- b. find $dL/dW2 = dL/dx2 * dx2/dW2$
 $= -2(o-x2)*x1$*
- c. find $dL/dW1 = dL/dx2 * dx2/dx1 * dx1/dW1$
 $= -2(o-x2) * W2 * x$*

Types of optimizers:-

Suppose your dataset has 1000 samples, your batch size is 10, your epochs is 5.

- a. Gradient Descent :- $W1^t = W1^{t-1} - learning_rate * dL/dW1^{t-1}$
We will have 5 updates only*
- b. Stochastic gradient descent
We will have 1000*5 updates*
- c. mini batch gradient descent:-
100*5 updates (1000/10 =100)*

All these use only the current gradient for update, this may be risky, so I use solutions which use multiple gradients instead of a single one.

-Adam

-RMSPProp

I am asked that with SGD I get 99% accuracy, should I move on to so called better optimizers?